

新規プロダクトを 高速で生み出す ハーネスエンジニアリング



Ryohei Ikegami

May 21, 2026 • 生成 AI 会 Vol.2@渋谷



池上 涼平

Ryohei Ikegami [@seanchas_t](#)

株式会社グッドパッチ

- **2025/05** AI デザインツール「Layermate」を個人開発でリリース
- **2025/10** グッドパッチに事業譲渡 / ジョイン
- **現在** Layermate に加えて、複数の AI 新規プロダクトの立案/開発に従事



Layermate

チャット形式でデザインを AI 生成する Figma プラグイン

<https://www.layermate.ai>

The screenshot displays the Layermate AI chat interface. At the top, a navigation bar includes the 'ChatMind AI' logo, links for 'Features', 'Pricing', and 'About', and a 'Get Started' button. The main content area features a hero section with the heading 'Chat with Intelligence, Unlock Possibilities' and a subtext: 'Experience the future of conversation with our AI-powered chat platform. Get instant answers, creative solutions, and intelligent assistance 24/7.' Below this are two buttons: 'Start Chatting Free' and 'Watch Demo'. A chat window titled 'ChatMind Assistant' is open, showing a conversation about the benefits of AI assistants. The assistant responds that AI assistants offer 24/7 availability, instant responses, personalized help, and can handle multiple tasks simultaneously, boosting productivity. To the right, a separate window titled 'Layermate | Create designs with AI chat' shows a generated design for an 'AI Chat App Landing Page'. The design includes a header with navigation, a hero section with a gradient background, and a call-to-action button. The chat window also displays the text: 'I'll create a modern landing page for an AI chat app with a hero section, features, and call-to-action.' and 'Generated Design'.

Landing Page

ChatMind AI

Features Pricing About Get Started →

Powered by Advanced AI

Chat with Intelligence, Unlock Possibilities

Experience the future of conversation with our AI-powered chat platform. Get instant answers, creative solutions, and intelligent assistance 24/7.

Start Chatting Free Watch Demo

ChatMind Assistant
Online

What are the benefits of AI assistants?

AI assistants offer 24/7 availability, instant responses, personalized help, and can handle multiple tasks simultaneously. They're great for boosting productivity! 🚀

Type your message...

Layermate | Create designs with AI chat

AI Chat App Landing Page

Create a landing page of an AI chat app

I'll create a modern landing page for an AI chat app with a hero section, features, and call-to-action.

Generated Design

I've created a comprehensive landing page for an AI chat app featuring:

Header: Clean navigation with logo, menu items, and CTA button

Hero Section: Eye-catching gradient background with:

- 1 2025 年: AI でコードはすぐ書けるように
- 2 ハーネスエンジニアリングで土台作り
- 3 機能の前に、何を作ったか
- 4 ハーネス自体がプロダクトになる

2025 年: AI で
コードはすぐ書けるように

AI コーディング が変えたこと

意図を伝えれば、AI が設計・実装・テストまで書く
人間は仕様確認と品質判断に集中できる

Layermate のリリースまでの実工数: **約 1 人月**

コーディング自体は桁違いに速くなった

でも、細かい指示が必要だったり

- AI が過去の文脈を忘れる
- 同じバグを何度も埋め込む
- 権限・セキュリティ・DB 変更は入念なレビューが必要

**AI コーディング は最初作るだけなら速いが、
低工数で運用し続けるためには環境整備が必要**

ハーネスエンジニアリング: スムーズなAI駆動開発の 土台作り

AI エージェント = 賢さ + 環境

AI エージェント = Model + Harness

Model (賢さ) LLM そのものの能力

Harness (環境) モデルが働くための土台 = 設計するのはこちら

設計軸	問い	例
コンテキスト	何を読ませるか	CLAUDE.md、設計ログ、アーキテクチャ文書
アクション	何を許可するか	権限、ツール、サンドボックス
フィードバック	どう失敗に気づかせるか	test、lint、typecheck、CI、drift detection
運用	どう継続的に改善するか	rules、PR の Why、設計ログ

先に作っておくと、何が嬉しいか

ハーネスの効用

壊さない	権限分離 / サンドボックス
間違いに気付ける	実 DB テスト / CI / 設定ズレ検出
レビュー疲れしない	rules / CodeRabbit が先に弾く
大雑把な指示で自走	CLAUDE.md / 階層化された文脈

人間レビューなし + 大雑把な指示で AI が自走 → 0→1 サイクルが爆速化

実践:
本格機能開発に取り掛かる前に
整備したハーネス

最初に整備したハーネス

技術スタック

Next.js / Hono / MobX / PostgreSQL + Drizzle / Firebase Auth / Google Cloud

領域	中身
コンテキスト	CLAUDE.md の階層化 (AI 向けの指示書)
失敗パターン	常に更新される rules
権限	RLS でテナント分離、SQL で宣言的に管理
テスト	実 DB で結合テストを厚く (トロフィー型)
スキーマ	SQLで宣言的に管理、DBとのズレを自動検出
実装後フロー	AI に PR 作成からレビュー対応まで任せる skill
チーム	ドキュメント + UI モックを全員で GitHub に集約

AI のミスを食い止める防御機構

層	防御方法	止まる例
事前に書かせない	CLAUDE.md / rules	AI が最初から正しい書き方を選ぶ
そもそも書きにくくする	権限分離 / ミドルウェア / 型	セキュアでない実装が書きにくい設計
動かして検出	実 DB を使った権限テスト	権限漏れを実行時に検出
マージ前に止める	CI (型 / テスト / 設定ズレ)	問題があれば merge できない
bot に 1 次レビュー	CodeRabbit	機械が先にチェック
人間は最後の判断だけ	人間レビュアー	判断が必要な箇所のみ

実践 1

機能ごとにコンテキストを分割し、段階的開示



フロントと API が同じ feature フォルダに同居 + CLAUDE.md も feature 単位

全部読ませず、段階的開示で AI に必要な範囲だけ渡す

実践 2 同じ指摘を 2 度繰り返さない

AI は、一度レビューで指摘したことを別セッションでまた忘れる

ルール	防いでいる事故
SQL の書き方	エスケープ忘れ / DB 接続の取り違い
API 呼び出し	戻り値チェック忘れ / 生 fetch 使用
状態管理	リアクティブ宣言忘れ / 不適切な更新
テストの書き方	権限テストの漏れ / エラーの握り潰し
自動生成ファイル	手編集の禁止 / レビュー対象外

レビュー指摘は随時 rules に移す skill を運用

実践 3

RLS で DB レベルのアクセス制御

アプリ側の権限チェックだけではない多重防御 → **DB 自身に権限を持たせる**

- **Row-Level Security (RLS)** でワークスペース単位の **テナント分離**
別テナントのデータは、クエリしても 0 行 (DB が弾く)
- **権限は SQL スキーマで宣言的に管理** — 全ポリシーを 1 ファイルに集約
「誰が何を読めるか」をアプリ全体で俯瞰できる

権限チェックをアプリだけでなく、DB に宣言する

実践 4 テストは「トロフィー型」で

E2E (Playwright) は遅く 不安定 → 結合テストをメインにする

層	ツール	厚み
結合	MobX store + Hono testClient + 実 DB	◎ 最厚
UI	happy-dom + testClient	○ 中
E2E	Playwright	△ 最小

MobX の store は描画せずテストできる → store + testClient で
「UI 状態 ← API」まで結合テストでき、E2E がほぼ要らない

ピラミッドではなくトロフィー — 真ん中（結合）を一番厚く

実践 5

スキーマは宣言的 SQL で管理する

DB スキーマを「あるべき姿」として SQL で宣言 → 差分は自動で埋める

schema/*.sql を編集（あるべき姿を宣言）

- migration を差分から自動生成
- TypeScript 型も自動再生成
- CI で「宣言 vs 実 DB」の drift を検出

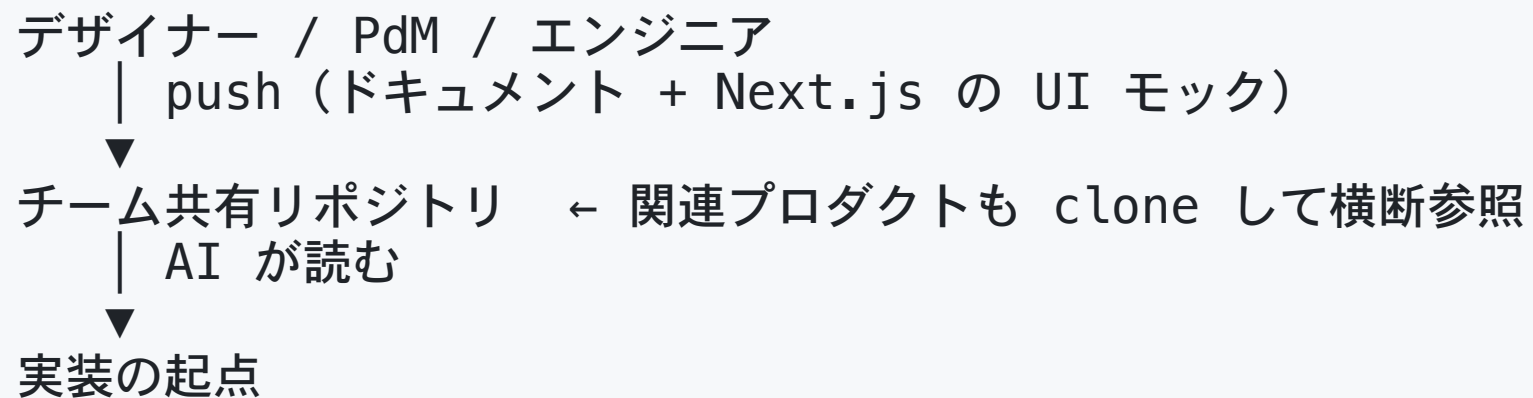
- スキーマ定義・マイグレーション・型が **ズレない**（単一の source of truth）
- AI がマイグレーションを忘れても、**CI がズレを見つけて止める**

実践 6 AI が PR 作成 → レビュー対応 → 知見保存する skill

ステップ	内容
1	ドキュメント反映 (差分を読んで CLAUDE.md / rules を更新)
2	PR 作成 (人間向け簡易説明 + AI向けの詳細な内容を説明文に含める)
3	CodeRabbit レビュー + 人間レビュー + CI を並行で監視
4	指摘の振り分け (修正 / Skip / 別 PR / 質問返し)
5	修正 + 返信 + 学びを rules に書き戻す
6	完了確認

PR作って、というだけであとは全自動

実践7 GitHub で全員がドキュメント管理



企画・仕様などのドキュメントも、**Next.js の UI モックも**、同じリポジトリで管理

push したドキュメント・モックが、そのまま実装の起点になる

未解決の課題（みなさんどうしてますか？）

CLAUDE.md / rules の肥大化対策

- 増え続ける rules の重複/矛盾の検出をどうする？
- 常時ロードをどこまで絞り、skill / 子 CLAUDE.md に逃がすか
- コンテキスト / rules を整理するメタハーネスの構築

AI の並行実装

- Git worktree は PORT / DB / env の分離がセットアップ煩雑
- 結局リポジトリを複数 clone するほうが楽な場面も
- 何並列までが人間の指示の限界か

AI への指示の変化

～2025（Layermate プラグイン 開発）

「ここはこう書く」と細かく指示
する必要があった

指示を出すコスト自体が重い

2026～（ハーネス整備後）

「この機能を追加して」で、文脈
読込・実装・権限・テスト・PR ま
で自走

人間は最初と最後だけ

まとめ

- 1 AI に機能を作らせる前に、AI が適切に作業できる環境（ハーネス）を作る
- 2 少ない人間のレビュー負担で、AI コードを通せる多層防御を組む
- 3 AI にプルリク作成 → レビュー対応までやらせる。学びは rules に書き戻す

細かい指示を減らして、AI が自走する開発へ

ハーネス自体が、
プロダクトになる？

HaaS = Harness as a Service

モデルそのものではなく、
モデルを取り巻く環境（ハーネス）をプロダクトとして売る

- モデルは汎用 LLM を利用（自前では作らない）
- 価値の源泉は、その周りの コンテキスト / ツール / UI / 安全性

HaaS の実例: ハーネスをプロダクトにする流れ

領域	プロダクト例
コーディング	Cursor / GitHub Copilot / Devin
コードレビュー	CodeRabbit
デザイン / プロト	v0 / Bolt.new / Lovable
業界特化	Harvey (法律) / Hebbia (金融)
CS / 業務自動化	Sierra / Salesforce Agentforce

どれも **モデル自体は提供せず、その周りの環境 (ハーネス) を売っている**

これ以外にも、業界特化のハーネスを提供するプロダクトは多数（国内にも続々）

今後のAIプロダクトは...

汎用 LLM だけでは届かない価値

精度	コンテキスト / スキル / 外部連携で、チャットに投げるより高い
使いやすさ	用途に即した UI で、チャット入力より自然
安全性	AI に渡す権限を絞り、危険な操作はブロック
継続性	使うほど rules / skills が育ち、賢くなる
連携性	既存の DB / SaaS / 内部ツールと繋がる

これからの AI プロダクトの正体は、ハーネス (かも?)